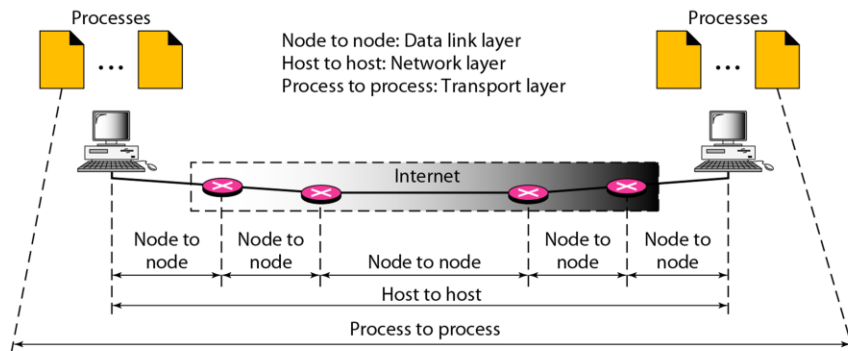


Process-to-Process Delivery: UDP, TCP

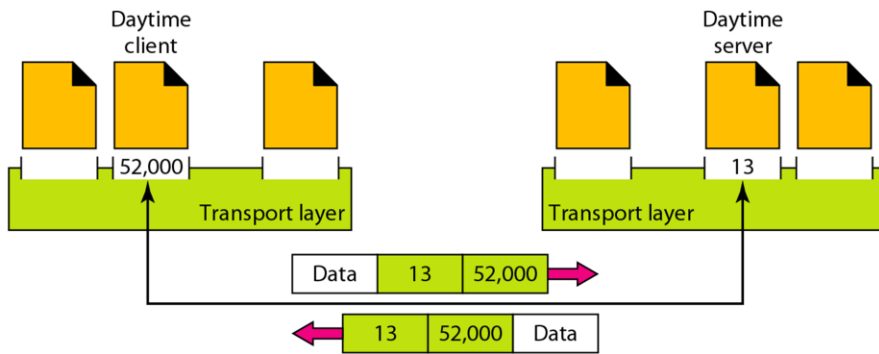
PROCESS-TO-PROCESS DELIVERY

The transport layer is responsible for process-to-process delivery—the delivery of a packet, part of a message, from one process to another. Two processes communicate in a client/server relationship.

Types of data deliveries

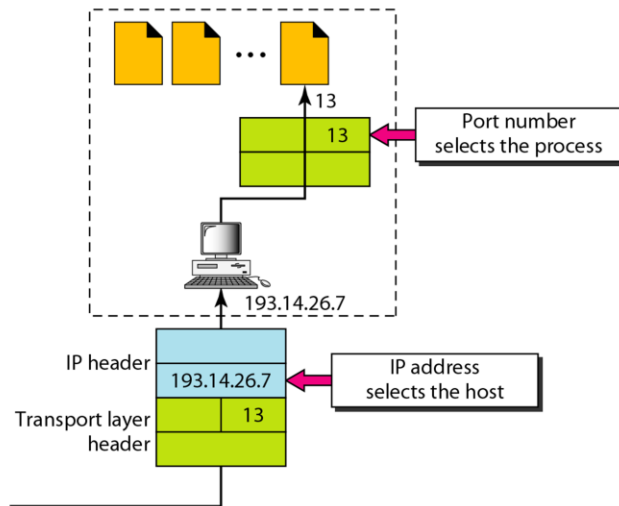


Port numbers

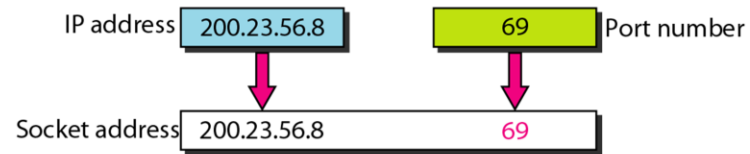


Protocol	TCP/UDP	Port Number
File Transfer Protocol (FTP)	TCP	20/21
Telnet	TCP	23
Domain Name System (DNS)	TCP/UDP	53
Dynamic Host Configuration Protocol (DHCP)	UDP	67/68
Hypertext Transfer Protocol (HTTP)	TCP	80
Network Time Protocol (NTP)	UDP	123
Simple Mail Transfer Protocol (SMTP)	TCP	25

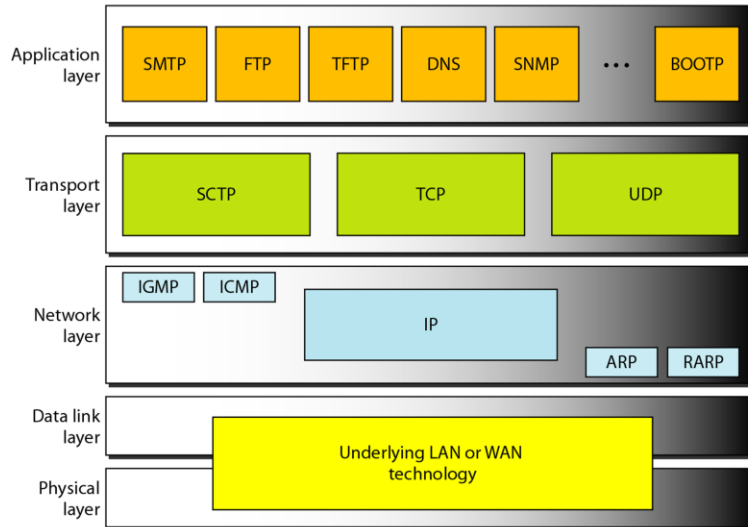
IP addresses versus port numbers



Socket address



Position of UDP, TCP, and SCTP in TCP/IP suite



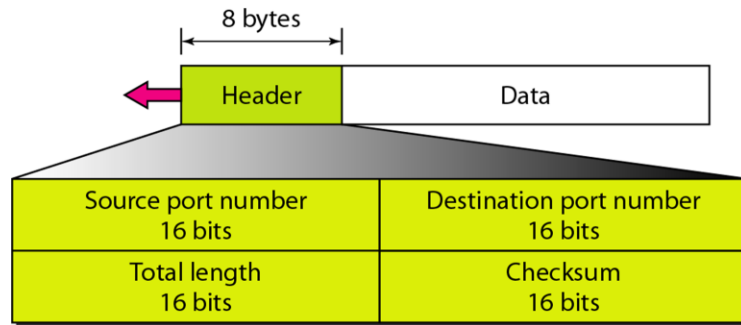
USER DATAGRAM PROTOCOL (UDP)

The User Datagram Protocol (UDP) is called a connectionless, unreliable transport protocol. It does not add anything to the services of IP except to provide process-to-process communication instead of host-to-host communication.

Well-known ports used with UDP

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	BOOTPs	Server port to download bootstrap information
68	BOOTPc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

User datagram format



TCP

TCP is a connection-oriented protocol; it creates a virtual connection between two TCPs to send data. In addition, TCP uses flow and error control mechanisms at the transport level.

Well-known ports used by TCP

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call



Note

The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number.



Example

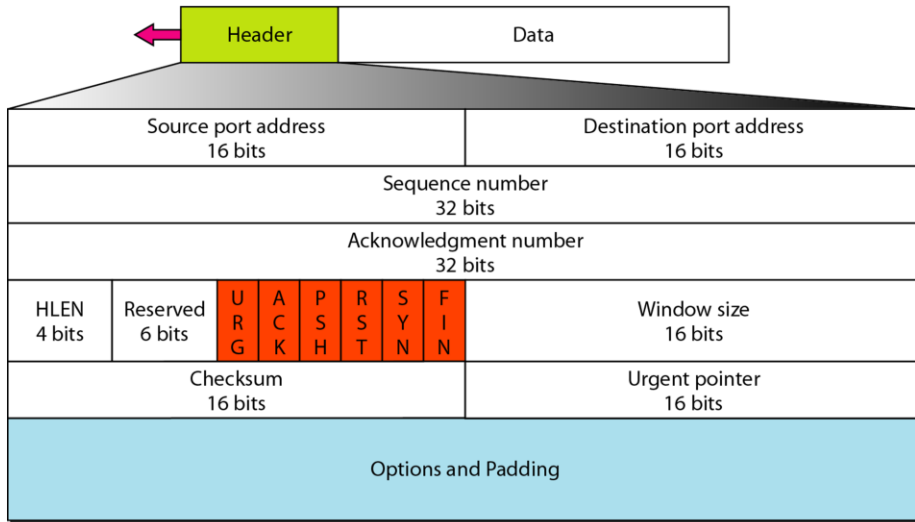
The following shows the sequence number for each segment:

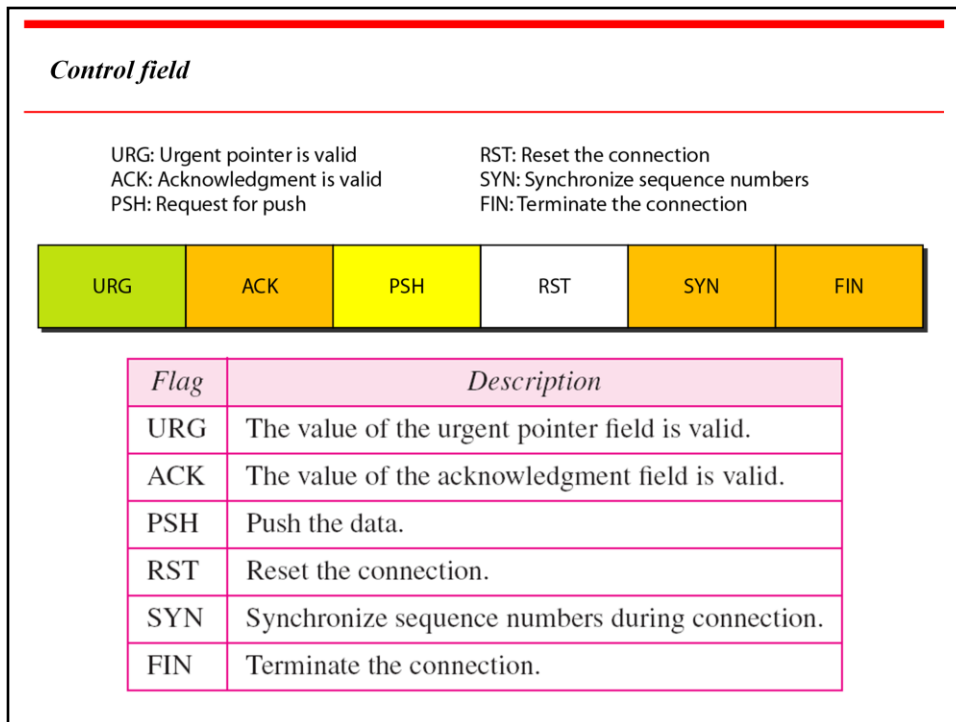
Segment 1	➡	Sequence Number: 10,001 (range: 10,001 to 11,000)
Segment 2	➡	Sequence Number: 11,001 (range: 11,001 to 12,000)
Segment 3	➡	Sequence Number: 12,001 (range: 12,001 to 13,000)
Segment 4	➡	Sequence Number: 13,001 (range: 13,001 to 14,000)
Segment 5	➡	Sequence Number: 14,001 (range: 14,001 to 15,000)

The value in the sequence number field of a segment defines the number of the first data byte contained in that segment.

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.

TCP segment format





Types of Flags:

Synchronization (SYN) – It is used in first step of [connection establishment](#) phase or 3-way handshake process between the two hosts. Only the first packet from sender as well as receiver should have this flag set. This is used for synchronizing sequence number i.e. to tell the other end which sequence number they should expect.

Acknowledgement (ACK) – It is used to acknowledge packets which are successful received by the host. The flag is set if the acknowledgement number field contains a valid acknowledgement number.

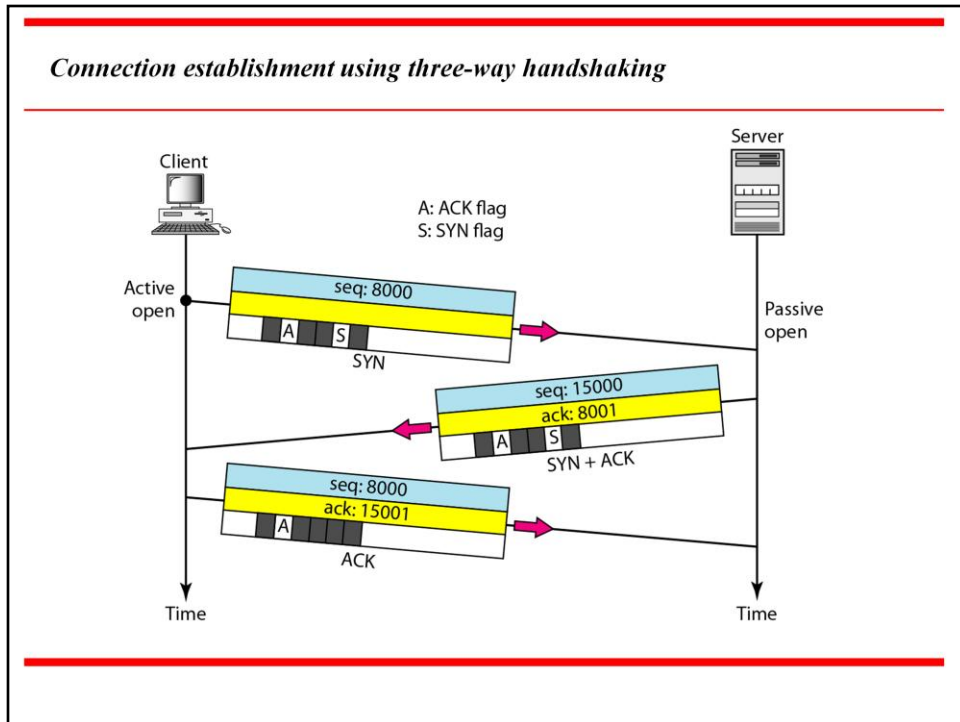
The receiver sends an ACK = 1 as well as SYN = 1 in the second step of connection establishment to tell sender that it received its initial packet.

Finish (FIN) – It is used to request for [connection termination](#) i.e. when there is no more data from the sender, it requests for connection termination. This is the last packet sent by sender. It frees the reserved resources and gracefully terminate the connection.

Reset (RST) – It is used to terminate the connection if the RST sender feels something is wrong with the TCP connection or that the conversation should not exist. It can get send from receiver side when packet is send to particular host that was not expecting it.

Push (PSH) – Transport layer by default waits for some time for application layer to send enough data equal to maximum segment size so that the number of packets transmitted on network minimizes which is not desirable by some application like interactive applications(chatting). Similarly transport layer at receiver end buffers packets and transmit to application layer if it meets certain criteria. This problem is solved by using PSH. Transport layer sets PSH = 1 and immediately sends the segment to network layer as soon as it receives signal from application layer. Receiver transport layer, on seeing PSH = 1 immediately forwards the data to application layer. In general, it tells the receiver to process these packets as they are received instead of buffering them.

Urgent (URG) –Data inside a segment with URG = 1 flag is forwarded to application layer immediately even if there are more data to be given to application layer. It is used to notify the receiver to process the urgent packets before processing all other packets. The receiver will be notified when all known urgent data has been received.



1. The client sends the first segment, a SYN segment, in which only the SYN flag is set.

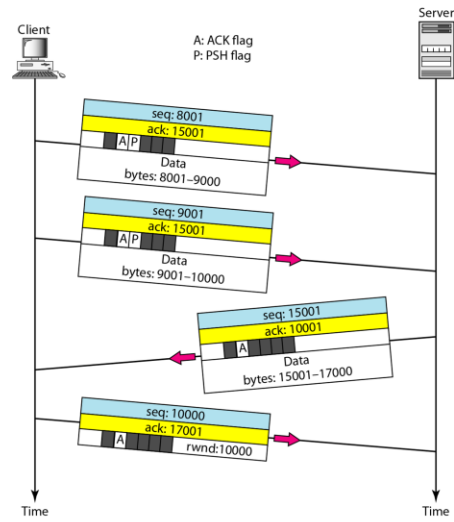
This segment is for synchronization of sequence numbers. The client in our example chooses a random number as the first sequence number and sends this number to the server. This sequence number is called the *initial sequence number (ISN)*. Note that this segment does not contain an acknowledgment number. It does not define the window size either; a window size definition makes sense only when a segment includes an acknowledgment. Note that the SYN segment is a control segment and carries no data. However, it consumes one sequence number because it needs to be acknowledged.

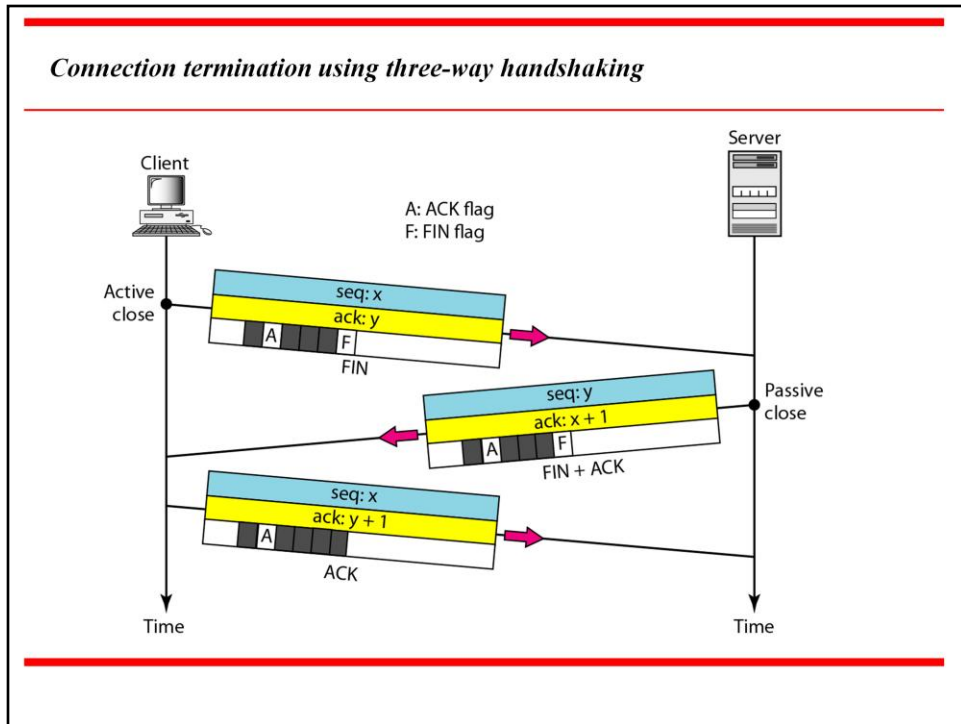
2. The server sends the second segment, a SYN + ACK segment with two flag bits set as: SYN and ACK.

This segment has a dual purpose. First, it is a SYN segment for communication in the other direction. The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client. The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag and displaying the next sequence number it expects to receive from the client. Because the segment contains an acknowledgment, it also needs to define the receive window size, *rwnd (to be used by the client)*. Since this segment is playing the role of a SYN segment, it needs to be acknowledged. It, therefore, consumes one sequence number.

3. The client sends the third segment. This is just an ACK segment. It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field. Note that the ACK segment does not consume any sequence numbers if it does not carry data.

Data transfer

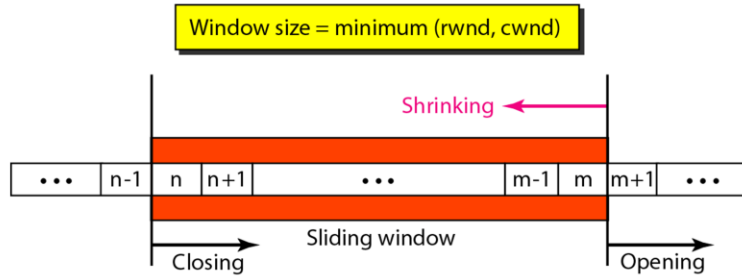




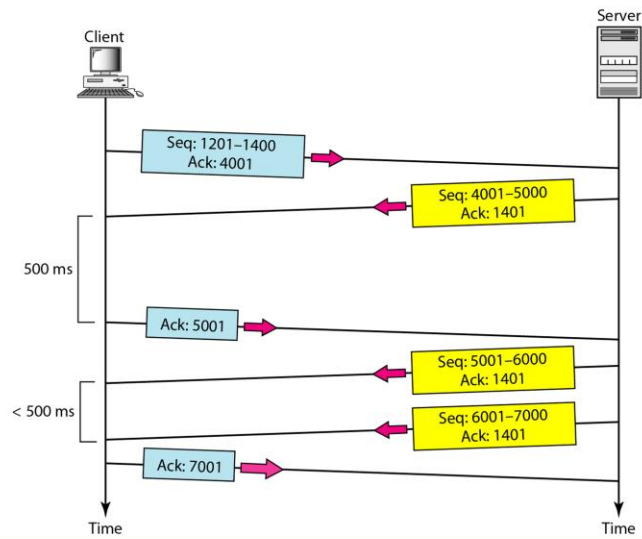
The FIN segment consumes one sequence number if it does not carry data.

The FIN + ACK segment consumes one sequence number if it does not carry data.

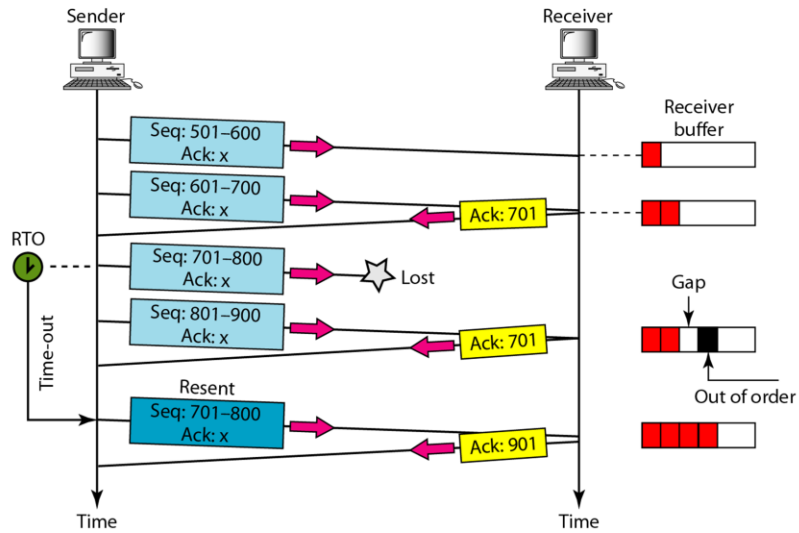
Sliding window



Normal operation



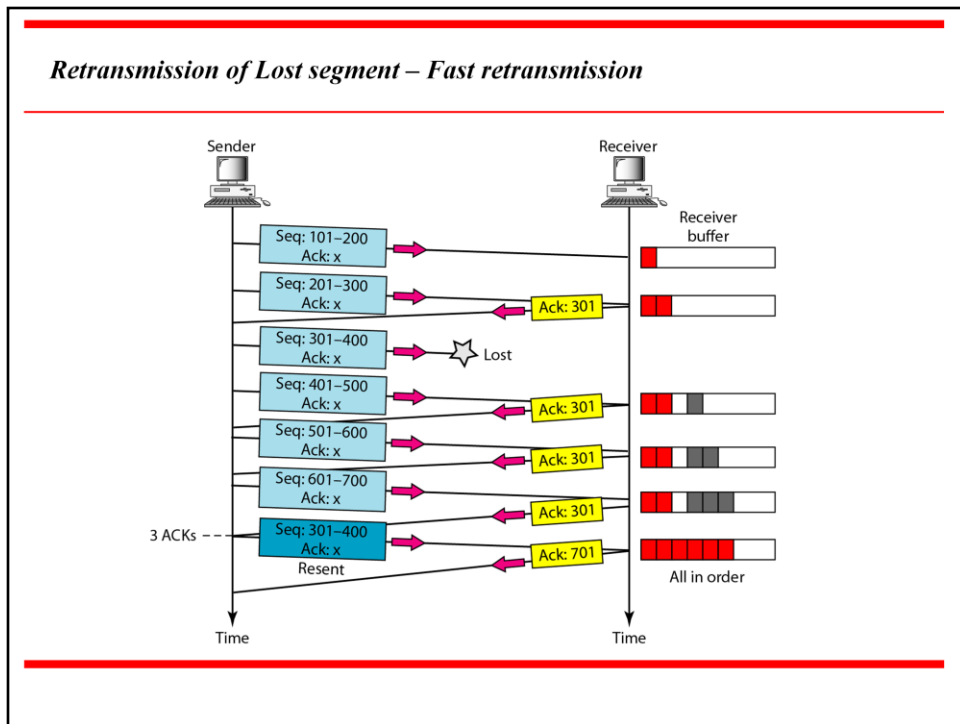
Retransmission of Lost segment – Timeout Approach





Note

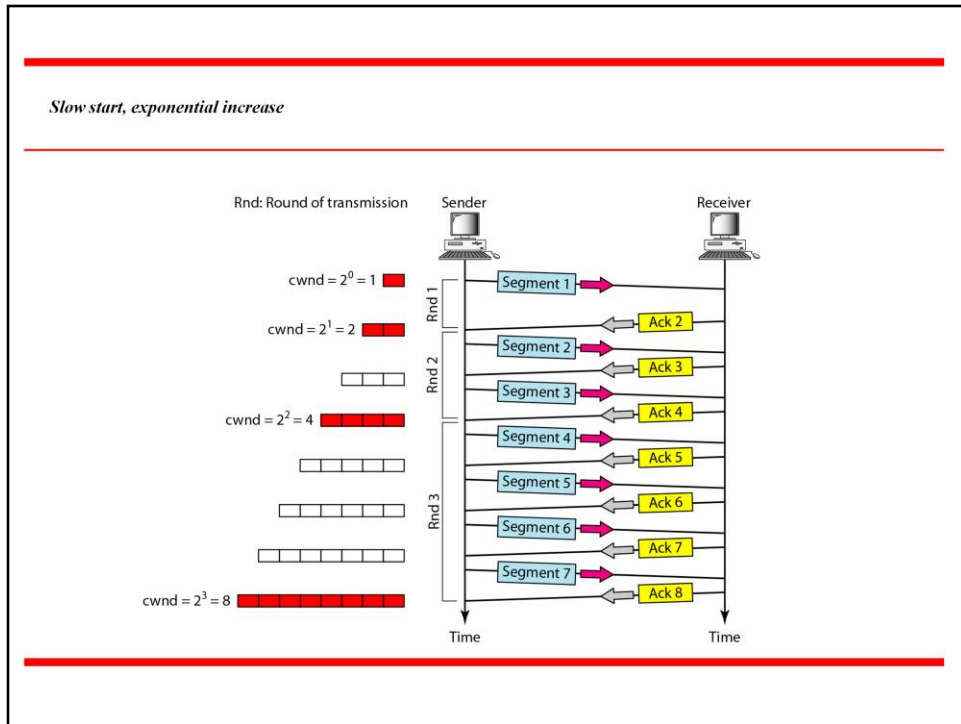
The receiver TCP delivers only ordered data to the process.



A new mechanism called *fast retransmit* was introduced that allows the [retransmission](#) of a lost packet even if the timeout for that packet has not expired. The fast retransmit mechanism does not replace the [timeout mechanism](#) of TCP, but it works in parallel to improve performance.

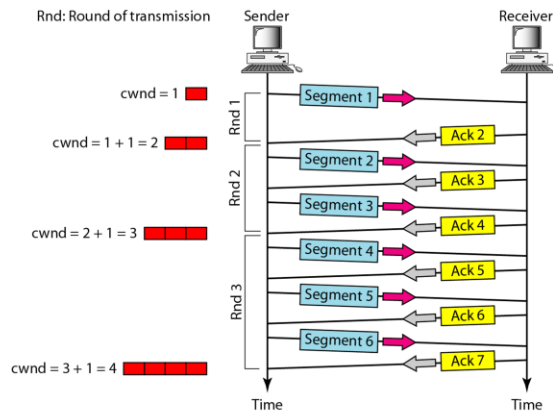
Every time a packet arrives out of order because the previous packet was lost or delayed, the receiver sends an acknowledgment that is the same as the one sent the last time. The subsequent transmissions of the same acknowledgment is called a *duplicate ACK*. When the sender detects a duplicate ACK, it knows that the receiver must have received a packet out of order, implying that the earlier packet was lost or delayed. To detect reliably the packets that are lost, the sender waits until it sees some number of duplicate ACKs before retransmitting the [missing packet](#). In practice, the sender waits until it has seen three duplicate ACKs, then retransmits the packet without waiting for its timer to expire.

TCP Congestion Control



In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Congestion avoidance, additive increase – AIMD (Additive Increase and Multiplicative Decrease)



In the congestion avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

An implementation reacts to congestion detection in one of the following ways:

- ❑ If detection is by time-out, a new slow start phase starts.
- ❑ If detection is by three ACKs, a new congestion avoidance phase starts.

TCP Tahoe

TCP Tahoe = Slow Start + AIMD + Fast Retransmit

Example

Suppose cwnd = 200 and packet loss occurs, then

ssthresh=cwnd/2= 100; cwnd will be reset to initcwnd value that is 10 (in Linux Kernel).

Again slow start will begin.

Once cwnd reached ssthresh, AIMD starts.

AIMD increases cwnd by 1 per RTT, so if packet drop occurs at cwnd = 125 then cwnd=10 and ssthresh=62 and Slow Start Restarts.

Slow Start phase: It lasts until the congestion window size reaches up to “**Slow Start Threshold (ssthresh)**“. The slow start algorithm doubles the congestion window size(cwnd) in one RTT. Initially, ssthresh is set to infinite(∞). Subsequently, it adapts depending on the packet loss events. When cwnd becomes equal to ssthresh, slow start stops. After that AIMD phase takeovers.

AIMD phase: It starts when the slow start stops. Additive Increase increases cwnd by 1 and Multiplicative Decrease reduces ssthresh to 50% of cwnd. Note that cwnd is ‘not’ reduced by 50% but ssthresh. This is the point to note that cwnd again resets to the initial window size (10 in Linux kernel).

Fast Retransmit phase: It is the loss detection algorithm. It is triggered by 3 duplicate acknowledgments. On packet loss detection, it resets cwnd to initcwnd.

TCP Reno

TCP Reno = TCP Tahoe + Fast Recovery

Example

cwnd=120, ssthresh= ∞

Packet loss detected using 3-duplicate ACK (aka Fast Retransmit).

cwnd will be reduced to 50%, cwnd=60 ssthresh will be new
cwnd, ssthresh=60

Now Reno has entered Fast Recovery Phase, it skips the slow start and AIMD takes over.

Whereas in Tahoe Slow Start restarts when packet loss is detected. This is the difference between these two.

Reno TCP

A newer version of TCP, called *Reno TCP*, added a new state to the congestion-control called the fast-recovery state. This version treated the two signals of congestion, time-out and the arrival of three duplicate ACKs, differently. In this version, if a time-out occurs, TCP moves to the slow-start state (or starts a new round if it is already in this state); on the other hand, if three duplicate ACKs arrive, TCP moves to the fast-recovery state and remains there as long as more duplicate ACKs arrive. The fast-recovery state is a state somewhere between the slow-start and the congestion-avoidance states. It behaves like the slow start, in which the *cwnd* grows exponentially, but the *cwnd* starts with the value of *ssthresh* plus 3 Maximum Segment Size (MSS) instead of 1.

Fast Recovery phase

It makes use of both RTO and Fast Retransmit. If the packet loss detection is triggered by 3 duplicate acknowledgments then it is a fast retransmit algorithm in action. On packet loss detection through Fast Retransmit, cwnd is reduced by 50% ($\text{cwnd} = \text{cwnd}/2$). 3 duplicate ACKs are received that means network is performing well because ACKs are being received which means packets are getting delivered to the receiver. So, cwnd is reduced by 50% in order to allow the network to come out of congested state. On packet loss detection through RTO, reset cwnd to initcwnd. If the RTO timer expires that means the network is badly congested. So, the cwnd has to be reduced to the initial value in order to recover the network from congestion.